2025 中山大学程序设计新手赛 题目讲解

ACMM 协会

2025-04-13

题目统计

本场比赛的题目预期难度排名为：

$$A < B < G < K < EI < M < D < CL < F < H < J$$

A. 时间复杂度

题目简述

给出五份代码，按照时间复杂度从小到大排序。

题解：本题需要分别考虑五份代码的时间复杂度。代码 A 和 C 的分析较为简单，如下所示：

```
// 代码 A，时间复杂度 O(n^2)
int k = 0;
for (int i = 1; i ⩽ n; ++ i)
  for (int j = i; j ⩽ n; ++ j)
    ++ k;
```

```
// 代码 C，时间复杂度 O(n^1.5)
int k = 0;
for (int i = 1; i ⩽ n; ++ i)
  for (int j = 1; j * j ⩽ n; ++ j)
    ++ k;
```

A. 时间复杂度

```
// 代码 B，时间复杂度 O(n log n)
int k = 0;
for (int i = 1; i ⩽ n; ++ i)
  for (int j = 1; i * j ⩽ n; ++ j)
    ++ k;
```

代码 B 的内层循环等价于从 1 到 $\lfloor \frac{n}{i} \rfloor$ 进行枚举，故瓶颈代码 `++ k;` 的执行次数为：

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor \approx \int_{1}^{n} \frac{n}{x} \, \mathrm{d}x = n \log n$$

故认为这一份代码的时间复杂度为 $O(n \log n)$。

A. 时间复杂度

```
// 代码 D，时间复杂度 O(Fib(n))
int fib(int n) {
  if (n ⩽ 1) {
    return 1;
  }
  return (fib(n - 1) + fib(n - 2)) % 998244353;
}

int result = fib(n);
```

代码 D 并未对已经计算的斐波那契数列值进行储存，故时间复杂度为 $O(\mathrm{Fib}(n))$。我们知道斐波那契数列的增长速率是指数级的。代码 E 储存了已经计算好的斐波那契数列值，故其时间复杂度是线性的。

A. 时间复杂度

```
// 代码 E，时间复杂度 O(n)
int arr[n + 1];
int fib(int n) {
  if (n ⩽ 1) {
    return 1;
  }
  if (arr[n] ≠ -1) {
    return arr[n];
  }
  return arr[n] = (fib(n - 1) + fib(n - 2)) % 998244353;
}
for (int i = 1; i ⩽ n; ++ i)
  arr[i] = -1;
int result = fib(n);
```

那么根据 $O(n) < O(n \log n) < O(n^{1.5}) < O(n^2) < O(\mathrm{Fib}(n))$，答案为 EBCAD。

B. 完美洗牌

题目简述

给出一副只包含点数的牌堆，包含 n 张牌（n 是偶数），在完美洗牌的规则下，求出在多少步后会回到初始状态。

题解：本题的关键在于发现在完美洗牌的规则下，一副牌在不超过 n 次洗牌后就一定会恢复原样。根据这一个性质实现暴力算法即可得到满分。时间复杂度为 $O(n^2)$。

关于为什么完美洗牌会快速恢复原样：若将牌堆的位置从 0 开始编号，可以发现在每次洗牌后，第 i 张牌的新位置 i' 满足 $i' \equiv 2i \pmod{n-1}$。又考虑到 2 和 $n-1$ 互质，而且 $i \equiv 0 \pmod{n-1}$ 对应的两个特殊情况无需考虑（首尾两张牌位置不变），可以根据简单的群论知识得到操作次数是 2 在模 $n-1$ 乘法群上的阶，也就不会大于 n。

G. 过河卒

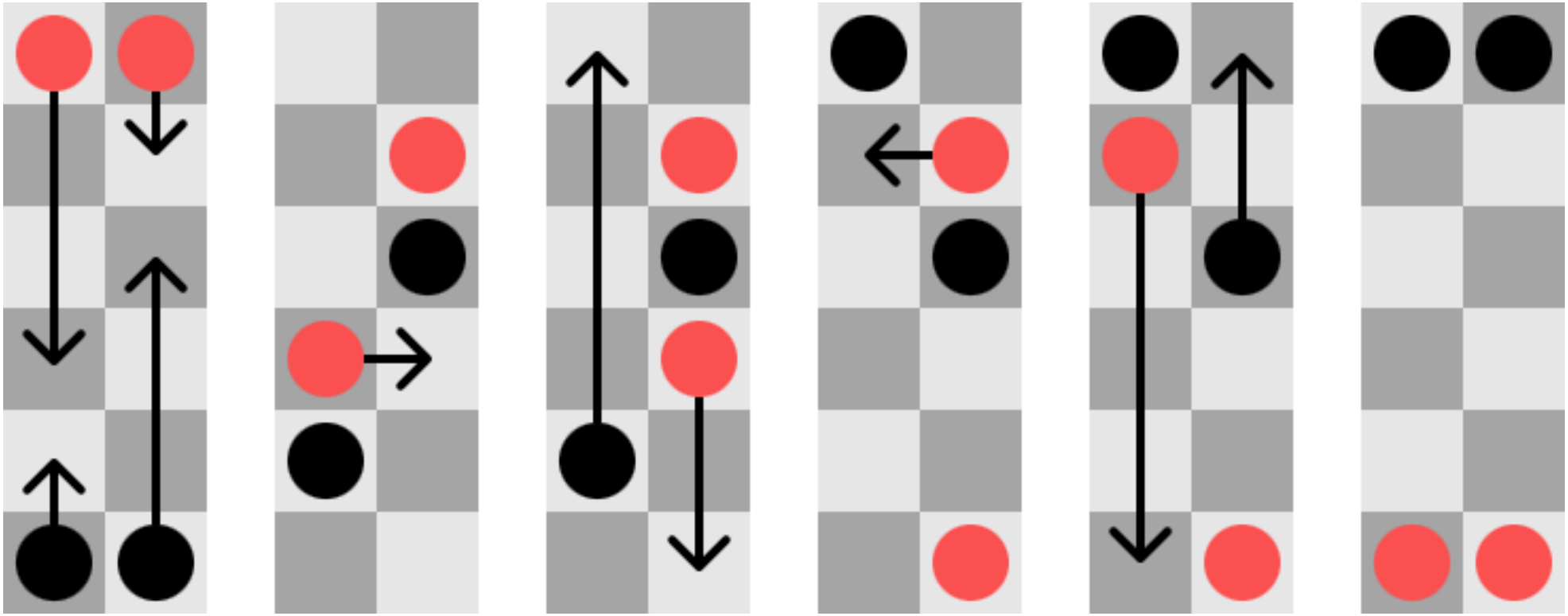
题目简述

给定一个 $n \times m$ 的棋盘，保证 $n \geq 6$ 且 $2 \leq m \leq 3$。棋盘的第 1 行有 m 个红子，第 n 行有 m 个黑子。现在要将所有黑子移动到第 1 行，所有红子移动到第 n 行。求出最少需要的步数。

题解：首先考虑到每个棋子都需要去到对侧，所需的最少操作数为 $2m(n-1)$。又因为 n 可以认为足够大，那么可以假设答案是 $2m(n-1)+f(m)$，其中 $f(m)$ 表示在 m 列的问题中棋子需要让步几次。实际上，通过观察样例即可得到 $f(2)=2, f(3)=4$，带入后即可得到本题的答案。时间复杂度为 $O(1)$。

下图是 $n=6, m=2$ 时对应的移动方案，可以轻松推广到本题的所有情况：

G. 过河卒



G. 过河卒

对这个问题的具体证明如下：

不妨考虑在最终的局面下，第 i 列的红子去到了第 p_i 列，而第 i 列的黑子去到了第 q_i 列。我们可以估计这一情况下所需步数的下界。考虑到在 $p_i = q_i$ 时，对应的两个棋子必须进行让路，产生至少为 2 的冗余步数。那么一个下界为：

$$L = \sum_{i=1}^{m} (n-1) + |p_i - i| + (n-1) + |q_i - i| + 2[p_i = q_i = i]$$

$$\geq 2m(n-1) + \sum_{i=1}^{m} (|p_i - q_i| + 2[p_i = q_i = i]) = L'$$

通过枚举或者观察可以发现 L 和 L' 的下界都是 $2m(n-1) + 2\lceil \frac{m}{2} \rceil$，恰好对应前面给出的构造。那么答案就是 $2m(n-1) + 2\lceil \frac{m}{2} \rceil$。

K. 分配系统

题目简述

对于一些初始容量固定为 S 的栈，需要通过“贪心选择剩余容量最大的栈”策略按顺序填入若干个大小不一的信息块，求出所需的最少栈数量。

题解：不难考虑二分栈个数，将原问题问题转换为可行性问题。为了确认 m 个栈是否足够，可以首先将最大堆初始化为 m 个相同的值 S，每次分配时从最大堆取出堆顶，判断容量是否足够并尝试进行分配。

考虑到栈个数显然不会超过数据块个数，那么时间复杂度为 $O(n \log^2 n)$。

E. Soft-取模

题目简述

给定正整数 x, l, r, d，依次将 x 对 $r, r-d, ..., l+d, l$ 取模，求出每次取模后得到的值的总和。

题解：不妨称让 x 的值发生变化的取模为“有效取模”，可以发现有效取模的次数不超过 3。以下是具体分析：

- 第一次取模可能是有效的，在取模后有 $x < r$；
- 考虑到模数以 d 为步长不断递减，那么在 $x \geq l$ 的前提下，第二次有效取模后应该有 $x < d$；
- 考虑到第二次有效取模后有 $x < d$，那么只有在 $l \leq x < d$ 时才会发生第三次有效取模，而这只可能发生在最后一次取模处。

E. Soft-取模

在明确了上述分析后，我们可以暴力找到下一次有效取模对应的模数，然后不断缩小问题规模即可。下面是可能的实现方案：

```
long long calc(int x, int l, int r, int d) {
  if (l > r)  return 0;
  if (x ⩾ r) return x % r + calc(x % r, l, r - d, d);
  if (x < l)  return (long long)((r - l) / d + 1) * x;
  // 找到最小的 a 使得 rr = r - a * d ⩽ x
  int a = (r - x + d - 1) / d;
  int rr = r - a * d;
  return 1ll * a * x + x % rr + calc(x % rr, l, rr - d, d);
}
```

时间复杂度为 $O(T)$。

I. 挑选座位

题目简述

给出一棵树，每个点有一个点权 a_x，需要支持两种操作：① 修改某个点的点权；② 查询一个点周围所有点的点权和。

题解：本题只需要维护一个点的父节点 f_x 和所有子节点的点权和 s_x，两个操作对应如下：

- 将节点 x 的点权修改为 y：在 x 不是祖先节点的情况下，将 s_{f_x} 加上 $y - a_x$，随后令 $a_x \leftarrow y$；
- 查询节点 x 周围所有点的点权和：计算 $s_x + a_x + a_{f_x}$ 即可。

时间复杂度为 $O(n + q)$。

M. 尖塔攻击

题目简述

在 $n \times m$ 的网格图上选定若干个可能相同的整点，并两两之间连线（相同点之间也会连线），找到两条线段使得它们的中点一样，并给出这个中点，或者判断无解。

题解：考虑到所有点都是整点，那么线段的中点必然满足横纵坐标都是某个整数的一半，而且依然落在网格图范围内。据此可以发现只会有最多 $(2n-1) \times (2m-1)$ 个可能的中点位置。

本题只需要找到两个线段，使得其中点重合，那么只需要枚举所有点对并储存中点位置，只要发现重复的中点就进行汇报。根据抽屉原理，中途计算的中点个数不会超过 $(2n-1) \times (2m-1) + 1$。可以使用二维数组判定避免其他数据结构带来的额外常数。时间复杂度为 $O(nm + k)$ 或 $O(nm \log(nm) + k)$。

D. 靶场

题目简述

一个圆的圆周被均匀分成 c 个瞄准点。一个人站在圆心处，面朝第 0 个瞄准点。有 n 个靶子按照时间顺序出现在某个瞄准点，而小 F 需要在靶子出现的一瞬间击中它们。给出小 F 每秒能转动的最多瞄准点数，求出他最多可以中靶几次。

题解：令 f_i 表示在第 i 秒射中了第 i 个靶子的前提下，在 i 秒及以前射中的靶子数量最大值。为了方便起见，可以假设第 0 个靶子放置在 0 号瞄准点处，和小 F 的初始方向相符。

D. 靶场

对于转移方程，只需要考虑上一个射中的靶子是什么即可。假设第 j 个靶子是上一个射中的靶子，且 $j < i$，那么可以得到如下表达式：

$$f_i \leftarrow \max_{0 \le j < i, h(i,j)} \left(f_j + 1 \right)$$

其中 $h(i,j)$ 表示“在射中第 j 个靶子后能够转身射中第 i 个靶子”命题的真假，而对于这个命题，可以算出两个瞄准点在圆周上的最近距离（也就是 $d(i,j) = \min\left(\left|a_i - a_j\right|, c - \left|a_i - a_j\right|\right)$），随后判断 $d(i,j) \le (j-i) * s$ 是否成立。考虑到本题 n 较小，暴力转移足以解决这个问题。

时间复杂度为 $O(Tn^2)$。

C. 游览路线

题目简述

对于一个带权无向连通图，一条路径的权值定义为每条边权值的按位与。给定起点和终点，求出路径的权值最大值。

题解：对于此类题目，考虑按位贪心，自高到低枚举答案的二进制位，并判断最终答案的这一位能否为 1。要想判定这一点，不妨假设当前需要检查的答案值为 m，那么只需要检查是否存在一条路径，使得路径上所有边的权值 w 均满足 $w \ \& \ m = m$，而这只需要将所有满足 $w_i \ \& \ m \neq m$ 的边临时删除，并判断 s 和 t 的连通性即可。

令 $V = \max a_i$，则时间复杂度为 $O(m \log V)$。本题允许你使用并查集判断两个点的连通性，此时的时间复杂度为 $O(m \log V \log n)$。

L. 提前修改

题目简述

给出一个数组 a_i，允许进行若干次修改，每次修改给出一个区间 $[l, r]$，对所有 $l \leq i \leq r$ 令 $a_i \leftarrow a_i \oplus i$。求出最少的修改次数，使得修改后满足 $\oplus_{i=1}^{n} a_i = 0$。需要判定无解。

题解：首先发现，操作是将一系列数字异或上一个值，而最终要求的结果也是异或和为 0，可以考虑首先计算 $A = \oplus_{i=1}^{n} a_i$，那么问题就等价于：在初始全为 0 的序列进行同样的修改操作，并使得最终的异或和等于 A。

不妨假设 $t = \lfloor \log_2(n) \rfloor$，也就是 n 的二进制最高位。可以注意到如下性质：

L. 提前修改

- 若 $A = 0$，则无需操作即可达到目的，下面讨论针对 $A \neq 0$ 的情况。
- 若 $A \geq 2^{t+1}$，说明 A 的二进制最高位大于 n 的二进制最高位，那么仅通过 $[1, n]$ 内的若干整数是无法异或得到 A 的，对应无解；
- 否则，令 $t' = \lfloor \log_2(A) \rfloor$，也就是 A 的二进制最高位。考虑到 $2^{t'} \oplus \left(A - 2^{t'}\right) = A$，并且 $A - 2^{t'} < 2^{t'} \leq 2^t \leq n$，故只需要最多两个 $[1, n]$ 内的数字（甚至不需要是区间）就可以异或出 A。在得到这一构造后，我们只需要尝试找到一步修改就能异或出 A 的方案。
- 一次修改异或出 A，等价于找到 $1 \leq l \leq r \leq n$，使得 $\oplus_{i=l}^{r} i = A$。在预处理前缀和数组 $p_i = \oplus_{k=1}^{i} k$ 后，条件等价于 $p_{l-1} \oplus p_r = A$。在枚举右端点 r 后，只需要查询之前是否出现过 $p_r \oplus A$ 即可找到 l。这部分实现较为开放。

时间复杂度为 $O(n \log n)$ 或 $O(n)$，根据具体实现而定。

F. 商店

题目简述

共有 n 种类型的商品，其中第 i 种类型的商品有 a_i 个，其中的 b_i 个是我们想要的。商店每天会从同类型商品未被购买的商品中随机抽取一个，只要这个商品是我们想要的商品，我们就会立刻购买。求出所有类型的商品都被购买的期望天数。

题解：考虑设 f_S 表示目前剩下 S 集合类型的商品时，全部购买完的期望天数。由于我们看到一个想要的商品就想要购买，那么可以通过接下来的方法计算答案。

首先可以计算第一次出现想要的商品对应的期望天数，这等价于不断重复随机一个以 $\frac{\sum_{i\in S} b_i}{\sum_{i\in S} a_i}$ 概率为真的命题，并求出其第一次为真的期望次数。根据概率和期望的关系，这个期望天数等于概率的倒数，也就是 $\frac{\sum_{i\in S} a_i}{\sum_{i\in S} b_i}$。

F. 商店

接下来考虑这个商品对应的类型。考虑到每个想要的商品被选中的概率没有差异，故对于其中某个类型 $k \in S$，这个商品恰好是 k 类型的商品的概率等于 $\frac{b_k}{\sum_{i \in S} b_i}$。那么我们可以得到转移方程：

$$f_S = \sum_{k \in S} \frac{b_k}{\sum_{i \in S} b_i} \left(f_{S-\{k\}} + \frac{\sum_{i \in S} a_i}{\sum_{i \in S} b_i} \right)$$

令 $f_{\{\}} = 0$ 后按照集合从小到大枚举并计算，最终答案为 $f_{[1,n]}$。

时间复杂度为 $O(n2^n)$。

H. 三仙归洞

题目简述

给出三个序列 a_i, b_i, c_i，需要支持两种操作：① 选定一个区间和两个序列，进行交叉互换；② 查询 a 数组的区间和。

题解：本题的关键在于如何处理交叉互换操作。首先，对于此类需要单次 $O(\log n)$ 查询区间和的题目，可以考虑使用线段树，而剩余的部分自然是考虑如何维护线段树上的信息和 tag。

考虑在线段树的节点处维护三个序列在对应区间的和，而对于交叉互换操作，则需要额外维护一个置换。置换在默认状态下等于 $\{1, 2, 3\}$，也就是不进行任何交换操作。

H. 三仙归洞

当我们需要交换 a 序列和 b 序列时，除了对维护好的三个和进行交换外，我们还需要将置换复合上 $\{2,1,3\}$，对应将前两个值交换。对于其他操作同理。在这种情况下，tag 的下传操作就可以同等的看作根据父节点的置换对子节点的信息进行交换或复合。这样我们就解决了交叉互换信息的维护问题。

时间复杂度为 $O(n \log n)$。

我们在场上发现了一种更加优秀的实现方案。具体的，考虑新建三棵线段树，通过一定的实现方法使得线段树上的节点可以交换，这样就只需要在交换操作时将两个线段树上对应的节点分别交换即可。

J. 高速公路

题目简述

给出一个带权无向连通图，需要回答若干次询问，每次询问包含一个点 u 和一个数量 c，需要求出最小的值 d，使得从 u 出发，仅经过边权不超过 d 的边可以到达至少 c 个点。

题解：本题存在两种做法。我们首先介绍在线做法（也就是能够做到立刻回答询问的做法）。

需要注意到，这一问题要求我们处理两个点之间的瓶颈路边权。两个点的瓶颈路边权 d 满足：在只经过边权小于 d 的边的情况下，二者不能互相到达，而 d 是满足这个限制的最小值。

J. 高速公路

根据最小重构树的性质，原图的最小瓶颈等价于任意一棵最小生成树上的最小瓶颈，而树上的路径是唯一的，相较于原图更容易处理，故可以考虑算出最小生成树，同时算出 Kruskal 重构树。Kruskal 重构树给出了一个重要的信息：两个点 u, v 的瓶颈路边权等于二者最近公共祖先对应的权值。

只需要在询问前处理 Kruskal 重构树每个子树的大小（不记录代表连接边的新点），然后在每次询问时倍增的从 u 向上跳，直到子树大小不小于 c 即可。

时间复杂度为 $O(m \log m + q \log n)$。

J. 高速公路

接下来是离线做法。考虑 Kruskal 算法的流程，我们按边权从小到大枚举边，并且将两个连通块合并。不妨将所有询问离线后分别储存在 u 连通块对应的堆中。在每次合并连通块的同时，可以对询问堆进行启发式合并，然后从堆中取出“c 不大于新连通块大小”的询问，并标记答案为当前枚举的边权。

使用可并堆实现启发式合并可以做到 $O(q \log q + m \log m)$，而使用 `set` 实现启发式合并可以做到 $O(q \log^2 q + m \log m)$。考虑到本题的限制，第二种做法可能会超时。